

A Training Data Recipe to Accelerate A* Search with Language Models

Anonymous ACL submission

Abstract

Recent works in AI planning have proposed to combine LLMs with iterative tree-search algorithms like A* and MCTS, where LLMs are typically used to calculate the heuristic, guiding the planner towards the goal. However, combining these techniques is not trivial: LM-based heuristics are quite weak, incurring a high computational cost without a significant performance improvement. Existing methods to learn these heuristics do not consider the requirements of the planner, and typically need a lot of compute. Thus, in this work, we propose a distribution to downsample training data by identifying relevant data points to learn a performant heuristic, while constraining computational costs. To arrive at this model, we disentangle the requirements of the planner, in our case A* search, from that of the language model to generalise on this task. Surprisingly, we find an overlap between their requirements; A* requires more accurate predictions on nodes near the goal, and LMs need the same set of nodes for effective generalisation. With these insights, we can quantify the contribution of each node towards accelerating A* search, and subsequently derive a training distribution for learning LM-based heuristics. Following a recent work, we conduct our experiments on two classical planning domains, maze navigation and sokoban, with two test splits per domain, and two conventional loss functions. We reduce the number of iterations required to find the solutions by upto 13 $\times$, with a wall-clock speed-up of upto 5 $\times$.

1 Introduction

Cognitive science reveals that human cognition can be understood as two closely collaborating systems (Kahneman, 2011). The fast system has high bandwidth, produces instantaneous responses, but is slow to adapt to novel problems. The slow system can solve novel problems, excels at logical reasoning, but works at a much slower speed. Recent

analyses suggest that due to their statistical learning nature, LLMs are analogous to fast systems; they perform poorly at problems that are novel (Wu et al., 2024) or require planning (Valmeekam et al., 2023). On the other hand, tree-search methods like A* (Hart et al., 1968) and variants (e.g., Korf 1985; Kocsis and Szepesvári 2006), are classic solutions to logical reasoning and planning, but are unable to learn from past experiences and limited in speed due to sequential dependencies. Combining these techniques remains an open problem.

We study the problem of using LLMs to learn A* heuristics, which are functions that estimate the distance from a search node to the goal state. However, ground-truth labels for training can only be obtained from successfully solved problems, rendering the training inefficient. Training LLMs is also computational demanding. Previous works typically train on all the search nodes, further exacerbating this issue. With this paper, we aim to improve the computational and data efficiency of heuristic learning by downsampling the training data from solved problems.

A common strategy in coreset selection is to select training data based on some measures of difficulty (Paul et al., 2021). In A*, a natural definition for difficulty is the distance to goal, as it is an estimate of how many decisions the search algorithm must make before reaching the goal, assuming a constant branching factor. Nonetheless, the A* + LLM setup contains two systems and each system may impose different requirements on training data, complicating optimization.

Hence, we first attempt to quantify the requirements of the A* system and the LLM respectively. We divide the search trajectory into three equally sized portions, the beginning, the middle, and the end, and test the effect of using each portion as training data. For the LLM, training on the last portion, where the data are easiest to fit, leads to better generalization than the other two portions.

Perhaps surprisingly, A* has a similar behavior; erroneous predictions on the end portion are the most detrimental.

Inspired by the empirical results, we devise a sampling strategy for LM training data that includes a higher proportion of search nodes near the end, than the beginning. This strategy outperforms baselines by finding solutions in an average of 8% lesser iterations, and, in some cases, even outperforms models trained with double the data.

Our contributions can be summarised as follows,

1. To the best of our knowledge, we are the first work to establish correlations between the nodes of the search tree, and their contribution to reducing the search length of A*. Further, we propose a mathematical model to quantify the contribution of each node.
2. We study the training data requirements for training LM-based heuristics to generalise and compare them to those of A*, and somewhat surprisingly, find an overlap between them.
3. Subsequently, we propose a distribution to subsample nodes from the search tree, that outperforms uniform subsampling on 4 test sets and 2 standard loss functions. Moreover, we demonstrate the efficacy of our method by scaling the LM upto 720M parameters.

2 Related Works

2.1 Learning Heuristics for Planning

Machine Learning Perspective In order to reduce the search length, learning for planning problems can be drawn back to (Yoon et al., 2006; Fern et al., 2011), which focused on learning domain-specific representations for planners through classical machine learning methods. Posing the task as a regression problem, learnt with neural networks was studied in (Arfaee et al., 2011; ús Virseda et al., 2013). Post their success, more recent works explored various neural architectures and objective functions for this problem (Chrestien et al., 2021; Groshev et al., 2018; Kirilenko et al., 2023). These works serve to prove the effectiveness of neural networks for learning strong heuristics in A* search, however, their methods are not designed by studying the problem from the planner’s perspective, i.e., by analysing the requirements of the planner to reduce the search length.

Planner Perspective Some works do look at the problem from this angle; (Yonetani et al., 2021; Vlastelica et al., 2019) reformulate each step of the planner as a differentiable function, and sequentially execute them, with the loss taken after the final step. Backpropagating through time enables them to create a search-dynamics aware heuristic. Similarly, (Speck et al., 2021; Orseau et al., 2023; Orseau and Lelis, 2021) learn heuristics by performing reinforcement learning on the actions taken by the planner. Such methods, while showing strong performance, incur a high training cost. In this work, we pose the problem as a regression task and take an alternate data-centric approach, where we analyse the planning algorithm, specifically A*, and find an optimal subset of nodes to train the neural network (in our case, language models). With this, we can constrain the training cost, while still learning a heuristic suitable for A*.

2.2 Heuristics with Language Models

LLMs in tree-search While in this work, we only study LMs being used as a heuristic in environments where possible actions are generated by rule-based methods, previous works have also explored using them as a world model, where the LLM generates an action given the state of the environment. (Yao et al., 2024) uses such a framework to build a tree and parse it with DFS/BFS, while (Hao et al., 2023) extends it to MCTS. The node to be expanded is selected with a heuristic computed by the LLM self-evaluating its generated actions. Such strategies are heavily dependent on the performance of this heuristic (Chen et al., 2024), which have been shown to be quite weak. In this work, we aim to improve such a heuristic by training it. Constraining our environments allows us to isolate the heuristic from the world model.

LLMs with external planners Besides a heuristic, LMs have been combined with external planners in various capacities. For instance, (Valmeekam et al., 2023) uses an LLM with the LPG planner (Gerevini et al., 2002), which iteratively corrects errors in a plan. Seeding LPG with an LLM plan has been shown to work better than a random plan. LLMs have also been used to translate tasks to formal languages for symbolic solvers like PDDL(Liu et al., 2023) and ASP(Yang et al., 2023). Combining such planners with LLMs has also been explored in dynamic settings to incorporate environment feedback (Guan et al., 2023; Dagan et al., 2023). While these works primar-

ily use off-the-shelf LLMs to improve symbolic planners, our work aims to train an LM.

Improving LM-based Heuristics (Shinn et al., 2024) proposed to improve LLM heuristics by incorporating failure states into the input prompt, thereby providing a semantic gradient signal away from such states. This has further been incorporated into tree-based frameworks (Zhou et al., 2023a). While such improvements are restricted to a particular instance, we consider the setting where the weights are updated for improvements across problems. An alternate line of work addresses the weak performance of LLMs for planning with CoT prompting. These works (Lehnert et al., 2024; Gandhi et al., 2024) propose to train on the traces of tree-search algorithms, and with bootstrapping methods, aim to implicitly learn an improved heuristic. On the contrary, we explicitly learn the heuristic by supervision.

2.3 Optimising Training Data

Coreset Selection involves pruning the training dataset to only contain important datapoints, without a significant drop in performance. While various works exist for LM pre-training (Paul et al., 2021; Marion et al., 2023; Abbas et al., 2023), to the best of our knowledge, we are the first work to study this in the context of heuristic learning. Our findings correlate with those of (Zhou et al., 2023b; Sorscher et al., 2022); easier data is required for learning in the low-data regime.

Curriculum Learning In identifying nodes required by the LM for generalisation, we categorise them as "easy", "medium" and "hard". Thus, our work also bears some similarities with curriculum learning methods, which propose a temporal ordering of data points of varying difficulty (Bengio et al., 2009). Some works demonstrate that learning from easier points first, helps learn complex concepts later (Xu et al., 2020), others find the reverse to be more beneficial (Maharana and Bansal, 2022). The aim of curriculum learning is to improve the performance on hard data points, whereas our aim is to reduce the search length of A*, regardless of the difficulty of the training data.

3 Preliminaries

3.1 A* Search

A* Search is a tree-based search algorithm that aims to find a path in an environment, given a start

node and a goal node by building a tree $\mathcal{T}$. The set of all nodes in the tree is defined as $\mathcal{N}$. For each node n , A* search keeps track of two values, (i) historical cost $g(n)$, which is the distance between the start node and n and (ii) heuristic $h(n)$ which is an estimate of the true distance $h^*(n)$ between n and the goal node. Each node also stores the domain-dependent environment state $s(n)$, as a result of an action. For the search, A* maintains two lists, the frontier list $PQ_{frontier}$ and the closed list PQ_{closed} , which are used to iteratively build the search tree. At the beginning, the closed list is empty and the frontier list is initialised with the start node (alias root node). The search is terminated when either the goal state is encountered, or $PQ_{frontier}$ is empty. Each iteration performs two steps, described below.

Selection This step picks the "most-promising" leaf node in the search tree. All leaf nodes are stored in the frontier list, implemented as a minimum priority queue on the node's fitness-value $f(n) = g(n) + h(n)$. If the state of the selected node is equal to the goal state, the search is terminated. Else, the next step is performed.

Expansion This step adds new children nodes to the selected node, thereby expanding the search tree. A child node is only added to the search tree if it is more promising than a future node (in the frontier list) or a previously expanded node (in the closed list). More formally, it is added to the search tree *iff* there does not exist a node with the same state in either the frontier, or the closed list, with a lower $f(\cdot)$ value. Finally, the selected node is moved from the frontier to the closed list.

We define the search length $\mathcal{S}$ of A* as the length of the closed list¹ after termination of the search. The use of $h(n)$ makes A* an *informed* search algorithm, significantly reducing the size of the search tree, when compared with uninformed search. The path from start to goal, defined as $\pi = (n_0, n_1 \dots n_l)$, is the sequence of l nodes from the root to the goal node. The start-to-goal path with minimum length is called the *optimal path*, denoted by π^* . A* guarantees that the resulting path will be optimal if the heuristic is *admissible*², i.e., $h(n) \leq h^*(n), \forall n \in \mathcal{N}$. It can be shown that with $h(\cdot) = h^*(\cdot)$ and non-trivial tie-breaking, A* will act as an optimal policy with $\mathcal{S} = |\pi^*|$. An

¹which in turn is equal to the number of iterations

²This can be easily proved by contradiction

inadmissible heuristic, however, does not guarantee sub-optimal solutions.

A* search is summarised in Algorithm 1.

3.2 Problem Definition

Our goal is train a language model θ , that, given a node n , can predict the residual between the perfect heuristic and its estimated heuristic value, $d^*(n) = h^*(n) - h(n)$. From a puzzle instance Γ , training instances can be derived for this task by considering each node in the search tree. For supervision, the perfect heuristic needs to be calculated, however, doing that for each node will require running A* again for all of them³. This becomes very expensive, particularly since the $\mathcal{S}$ for a single puzzle can be $> 10k$. Therefore, we only consider nodes on the optimal path, since calculating their $h^*(\cdot)$ is trivial : for any node $n_j \in \pi^*$, $h^*(n_j) = |\pi^*| - j$. This is also the conventional setting used in previous works (Chrestien et al., 2021; ús Virsedá et al., 2013). Therefore, the training sequences $\mathcal{X}$ are given by $\mathcal{X} = \bigcup_{\pi_i^* \sim \Gamma_{i=0}^N} \{(n_j, d^*(n_j)), n_j \in \pi_i^*\}$.

3.3 Loss functions

We consider two losses to train θ , the L2 loss,

$$\mathcal{L}_{L2} = (\theta(n) - d^*(n))^2 \quad (1)$$

and the language modelling objective, given by,

$$\mathcal{L}_{LM} = -\log p(d^*(n)|\theta(n)) \quad (2)$$

We use encoder-decoder transformers for our experiments, and add a regression head ϕ_{L2} on the decoder that predicts $d^*(n)$ given the $< bos >$ token while training with $\mathcal{L}_{L2}$. With $\mathcal{L}_{LM}$, the (pre-trained) language model head ϕ_{LM} is used. The encoder input consists of a prompt, along with the details of a node.

3.4 Inference

Inference involves leveraging the trained LM in A* search. This is done by adding an LM inference with the previous heuristic calculation. During the expansion step, the child nodes are converted into a prompt and collated in a batch, which is used to predict $d(n)$. This is added to $h(n)$, and search continues. Notably, only a single forward pass is performed per expansion. Additionally, we cache these prompts, such that if a state is revisited in

³Running Dijkstra’s algorithm is intractable for complex domains, like sokoban

a another node m , $d(m)$ can simply be retrieved, rather than predicted. For θ trained with $\mathcal{L}_{LM}$, we perform top-k decoding, with $k = 5$, along with self-consistency(Wang et al., 2022), predicting 3 sequences, as this works slightly better in practice.

3.5 Domains

We conduct our experiments on two domains, denoted in Figure 2 and Figure 3. All experiments are conducted on two test splits, the IID and OOD.

Maze Navigation is a standard maze puzzle that involves finding an unobstructed path from the start to the goal state. The state of a node $s(\cdot)$ is characterised by the position of the player on the board. The admissible heuristic function used in the training data (and reference solutions) is the manhattan distance between the player and the goal positions. Training and validation is performed on sequences derived from mazes of size 20×20 . The IID test split thus consists of mazes of the same size, while the OOD split consists of mazes of size 30×30 .

Sokoban is a japanese puzzle game, that involves a player pushing one or more boxes to fixed docks. This puzzle is considerably harder than maze, since a few wrong moves can lead to deadlocked situations, rendering the puzzle unsolvable. The state of a node $s(\cdot)$ is characterised by the position of the player on the board, and the position of the boxes. Note that all boxes and docks are identical. The admissible heuristic function used is the sum of the minimum manhattan distance between the player position and a box, and the sum of manhattan distances between the boxes and their assigned docks. Boxes are assigned to docks by solving the minimum cost assignment problem with the Hungarian algorithm. Training and validation is performed on 2 box problems, while testing is done on a mixture of harder 2, 3 and 4 box problems.

The exact data generation process is described in subsection A.1.

3.6 Metrics

We modify the metrics defined in (Lehnert et al., 2024), (i) inverse-length-ratio(ILR) to measure the differences in the search length, (ii) success weighted by cost(SWC) to measure the differences in solution length and (iii) Optimal %, to measure the percentage of puzzles solved optimally. ILR measures the average inverse ratio between the search length $\hat{S}$ of an A* solution, to that of a

Set with $h^*(\cdot)$	σ	ILR-on-solved	ILR-on-optimal	SWC	Optimal %
All	-	2.7356	2.7356	1.0000	100
Initial	2	1.7314	1.7717	0.9896	84.9
Middle		1.8911	1.9309	0.9908	86.4
End		2.2248	2.2617	0.9919	87.9
Initial	4	1.0842	1.1912	0.953	46.1
Middle		1.1604	1.2924	0.9516	46.2
End		1.5439	1.7389	0.9520	46.3
Initial	6	0.8579	0.9827	0.9229	28.6
Middle		0.9192	1.0811	0.9232	29.3
End		1.2157	1.5287	0.9202	28.1

Table 1: Experimental Results with the oracle heuristic.

reference $\mathcal{S}^*$. It is formulated as,

$$ILR = \frac{1}{N} \sum_{i=0}^N \frac{\mathcal{S}_i^*}{\mathcal{S}_i} \quad (3)$$

ILR can be averaged over various sets, (i) ILR-on-solved is averaged over all puzzles in the test set and (ii) ILR-on-optimal is averaged over all puzzles whose solutions are optimal. Suboptimal solutions, found with inadmissible heuristics, are often discovered with lesser $\mathcal{S}$, and larger ILR.

SWC measures the average inverse ratio between the start-to-goal path length $|\tilde{\pi}|$ of an A* solution, to that of an optimal reference, denoted by $|\pi^*|$. It is formulated as,

$$SWC = \frac{1}{N} \sum_{i=0}^N \frac{|\pi_i^*|}{|\tilde{\pi}_i|} \quad (4)$$

Additionally, to measure computational cost, we also propose a new metric, Inverse-Time-Ratio, which is defined as the average inverse ratio between the wall-clock time of an A* solution $\tilde{WT}$ and a reference solution WT^* , formulated as,

$$ITR = \frac{1}{N} \sum_{i=0}^N \frac{WT_i^*}{\tilde{WT}_i} \quad (5)$$

ITR can be averaged over the entire test set (ITR-on-solved), or on the problems solved optimally (ITR-on-optimal).

4 Disentangling A* and Heuristic Learning

4.1 Requirements of A*

When learning a heuristic with a language model, given a search node n , we predict the difference $d^*(n) = h^*(n) - h(n)$ (Kirilenko et al., 2023), where the final heuristic used during the search is $h(n) + d^*(n)$. However, as with any neural network, there is some error in the prediction, resulting in a value close to $d^*(n)$, and in turn close to

$h^*(n)$. We aim to study two aspects, (i) how this error affects $\mathcal{S}$, and (ii) how it affects optimality of the solution. To achieve this, we perform experiments with an oracle heuristic, that always uses $h^*(n)$ during the search. We artificially introduce error into this heuristic to study these effects. The error is introduced in different sections of the tree to identify correlations between the sets of nodes with low error, $\mathcal{S}$ and optimality.

Experiment Here, we use to use the perfect heuristic $h^*(\cdot)$ in certain sections of the tree, and $h(\cdot) \sim N(h^*(\cdot), \sigma)$ in other sections. $h^*(\cdot)$ is calculated offline, by running Dijkstra’s algorithm on the maze, starting from the goal. The tree is divided into three sets - *initial*, *middle* and *end*. A node n is placed in the *initial* set if $g(n) < |\pi^*|/3$, in the *middle* set if $|\pi^*|/3 \leq g(n) < 2|\pi^*|/3$, and in the *end* set if $g(n) \geq 2|\pi^*|/3$. *all* refers to all the nodes in the tree. We perform an ablation, where we use the perfect heuristic in each of these sections, and introduce noise in the others. The upper limit of each metric is given by using $h^*(\cdot)$ in *all* sections, albeit with trivial tie-breaking. Results are shown in Table 1.

As evidenced by the scores of *all*, low error on all the sets leads to the best performance. However, achieving that with a learned heuristic is difficult (discussed later). Hence, we try to identify smaller sets with the best scores. Amongst these, with the same σ , using $h^*(\cdot)$ on nodes in the *end* set performs the best on both, ILR-on-solved and ILR-on-optimal, than using it on any other set. Moreover, using $h^*(\cdot)$ in the *middle* and *initial* sets gives diminishing returns, particularly with higher σ .

While there does not seem to be a trend between SWC and Optimal % amongst the three sets, both these metrics go down with increasing σ . This is not surprising, since the heuristic will be less admissible, thereby increasing the possibility of finding suboptimal solutions.

With these experiments, we make the following conclusions, (i) ILR can be improved by having low error in the heuristic for nodes near the goal, (ii) there are diminishing returns to ILR when the heuristic has low error for nodes further away from the goal, and (iii) optimality is directly related to the error in the heuristics.

Quantifying a Node’s Contribution to $\mathcal{S}$ Based on the above observations, we propose to quantify the contribution of a node to reducing the search



Figure 1: Validation MAE of a model trained on splits containing nodes from the *initial*, *middle*, *end*, *all* sets, and their corresponding exclusion sets. A lower value shows better generalisation.

Train Split	Test Split	ILR-on-solved	ILR-on-optimal	SWC	Optimal %	ILR-on-solved	ILR-on-optimal	SWC	Optimal %
		Maze				Sokoban			
All	IID	1.5666	1.5654	0.9972	97.60	8.3800	8.8785	0.9761	73.94
Initial		0.9101	0.9101	1.0000	100.0	0.6658	0.6661	0.9967	93.66
Middle		0.8370	0.837	1.0000	100.0	0.9710	1.0049	0.9901	83.80
End		1.2081	1.2033	0.9974	97.40	3.0312	3.0642	0.9965	93.66
~ Initial		1.2117	1.2132	0.9989	99.00	6.1912	6.5422	0.9862	82.04
~ Middle		1.6053	1.6151	0.9907	92.80	9.7389	9.9559	0.9578	56.69
~ End		0.9202	0.9202	1.0000	100.0	2.8397	2.9638	0.9854	80.28
All	OOD	1.3320	1.3309	0.9965	96.00	11.1967	11.7906	0.9815	74.46
Initial		0.8193	0.8193	1.0000	100.0	0.5940	0.5917	0.9956	90.12
Middle		0.8059	0.8059	1.0000	100.0	0.8148	0.8399	0.9904	84.34
End		1.1018	1.1055	0.9957	95.40	2.7465	2.7721	0.9986	96.39
~ Initial		1.0581	1.0594	0.9992	98.80	9.2832	9.8333	0.9893	83.86
~ Middle		1.2476	1.2360	0.9950	94.40	16.3567	18.1764	0.9650	61.45
~ End		0.9198	0.9198	1.0000	100.0	2.9484	3.0910	0.9854	78.07

Table 2: Experimental results by training on different splits of data, demonstrating the importance of the *end* split for generalisation to A* search on both, maze and sokoban.

length of a puzzle as,

$$\mathcal{C}(n) = \log\left(\frac{|\pi^*|}{|\pi^*| - g(n)}\right) \quad (6)$$

$\mathcal{C}(\cdot)$ assigns the highest value to nodes closer to the goal, while accounting for the diminishing returns to reducing search length of nodes further from it. Note that while there can be nodes with $g(n) \geq |\pi^*|$, they are never added to the tree, thus $\mathcal{C}(\cdot)$ is not defined for them.

4.2 Requirements for Heuristic Learning

We have shown that in order to reduce the search length of A*, we need to predict the heuristic more accurately for nodes closer to the goal. Here, we explore how training on these nodes affects generalisation to other nodes, and in turn model performance on ILR. Specifically, we create four training splits from the puzzles by uniformly sampling

nodes (on the optimal path) from the *initial*, *middle*, *end* and *all* sets. Additionally, we also train on *exclusion sets*, $\sim$ *initial*, $\sim$ *middle* and $\sim$ *end*, which excludes the corresponding set from the nodes to be sampled. For instance, $\sim$ *initial* is trained on data sampled from the *middle* and *end* sets. All training splits have the same size. We train the model on both domains with the following evaluations, (i) mean average error(MAE) on validation splits containing nodes from each of the aforementioned splits and (ii) and improvements in search length achieved by models trained on them. While (i) demonstrates the generalisation capability of the model, (ii) serves to extend the observations of the oracle heuristic to the LM.

Figure 1, show that each split generalises the best to itself, but shows poor generalisation to the others. The *all* set achieves the best generalisation to each split. Comparing the performance on ILR,

Train Split	Test Split	ILR-on-solved	ILR-on-optimal	SWC	Optimal %	ILR-on-solved	ILR-on-optimal	SWC	Optimal %
		Maze				Sokoban			
Model : $\mathcal{L}_{L2}$									
Full-data	IID	1.6739	1.6756	0.9967	97.0	11.6416	12.5933	0.9834	79.93
$\mathcal{X} \sim \mathcal{U}(n)$		1.5666	1.5654	0.9972	97.6	8.3800	8.8785	0.9761	73.94
$\mathcal{X} \sim \mathcal{D}(n, \tau)$		1.7029	1.7035	0.9958	96.6	10.2077	10.8168	0.9808	75.7
Full-data	OOD	1.2755	1.2730	0.9967	95.8	14.7093	15.2655	0.9847	77.83
$\mathcal{X} \sim \mathcal{U}(n)$		1.3320	1.3309	0.9965	96.0	11.1967	11.7906	0.9815	74.46
$\mathcal{X} \sim \mathcal{D}(n, \tau)$		1.3365	1.3354	0.9964	95.0	13.7706	13.7546	0.9828	77.11
Model : $\mathcal{L}_{LM}$									
Full-data	IID	1.4752	1.4902	0.9925	94.0	9.2978	10.4147	0.9594	60.92
$\mathcal{X} \sim \mathcal{U}(n)$		1.4979	1.5070	0.9897	92.2	7.1347	7.4233	0.9607	61.62
$\mathcal{X} \sim \mathcal{D}(n, 10)$		1.5517	1.5628	0.9897	92.2	7.8141	8.0857	0.9614	59.86
Full-data	OOD	1.2448	1.2467	0.9965	96.2	14.8513	16.1940	0.9645	61.45
$\mathcal{X} \sim \mathcal{U}(n)$		1.1869	1.1769	0.9925	92.8	12.4740	14.7325	0.9500	48.92
$\mathcal{X} \sim \mathcal{D}(n, 10)$		1.2426	1.2436	0.9940	93.0	13.3144	12.4565	0.9558	52.53

Table 3: Experimental results by sampling from the proposed $\mathcal{D}(n, \tau)$ distribution. Best scores are in **bold**. For $\mathcal{L}_{L2}$, the value of τ differs on each domain, with $\tau = 2$ for maze and $\tau = 0.8$ for sokoban.

the LM heuristic performs the best when trained on nodes from the *end* set, when compared to the performance of *middle* and *initial*. However, this is still lesser than the performance of *all*. This confirms the trends observed by the oracle heuristic generalise to the LM.

Amongst the exclusion sets, we observe that $\sim end$ achieves the worst generalisation and ILR performance on both domains and test splits. The generalisation between the other two sets is mixed, with $\sim middle > \sim initial$ on maze, and the reverse on sokoban. However, both their validation metrics are below that of *all*⁴. With these insights, we conclude that the *end* set is necessary for generalisation and ILR performance, and that the training data should be augmented with nodes from the other two sets, for enhanced generalisation. A higher proportion of nodes from the *end* set reduces the validation error on that set (albeit, at the cost of others), which is required to reduce search length.

Nodes in the *end* set can be considered "easier" than nodes in the other sets, since the LM requires fewer decision steps to predict their heuristic. We can thus also conclude that in our low-data regime, easier points are necessary for generalisation.

5 Proposed Solution

With these observations, we can conclude that the *end* set has the most contribution to reducing the search length in A^* , and is required for enhanced generalisation with LM-based heuristics. Moreover, augmenting the training set with points from the *initial* and *middle* set also leads to better gen-

eralisation, as indicated by the increased validation scores. Combining these, instead of sampling nodes on the optimal path uniformly, we propose to sample from the distribution,

$$\mathcal{D}(n, \tau) = \text{SoftMax}\left(\frac{1}{\tau} \mathcal{C}(n)\right), \forall n \in \pi^* \quad (7)$$

where τ denotes temperature. Increasing τ increases the hardness of the training dataset.

6 Experiments

Experimental Settings Sampling from $\mathcal{D}(n, \tau)$ is compared with sampling from a uniform distribution $\mathcal{U}(n)$. We also add a *full-data* baseline, which does not subsample and trains on all nodes on the optimal path. Both $\mathcal{L}_{L2}$ and $\mathcal{L}_{LM}$ are compared in Table 3. The full-data baseline is trained on 26.3k nodes for sokoban and 22.3k for maze, while the models with sampling are trained on 8k and 12k nodes, respectively. Subsampling involves sampling equal number of nodes from each puzzle. All models are initialised with code-t5-small with 60M parameters, unless otherwise mentioned. Hyperparameter details are mentioned in subsection A.4

Results Sampling from $\mathcal{D}(n, \tau)$ consistently outperforms uniform sampling on ILR by an average of 4.4% on maze, with a larger improvement of 12.5% on sokoban. On maze, $\mathcal{L}_{L2}$ also outperforms the full-data baseline, which is trained on 46.5% more data points. The results on SWC and Optimal % are mixed; while both models find shorter solutions when trained with nodes sampled from $\mathcal{D}(n, \tau)$ on sokoban, the same cannot be said for maze. Regardless, it can be concluded that

⁴The high ILR of $\sim middle$ is therefore not representative

both subsampling methods give similar-cost, near-optimal solutions. Interestingly, training on all the data gives higher performance on optimality metrics, which could be a result of lower validation error, due to more training data.

Training Target Between $\mathcal{L}_{L2}$ and $\mathcal{L}_{LM}$ models, the former consistently outperforms the latter on the IID test split, while on OOD, the results are mixed, with $\mathcal{L}_{LM}$ being slightly better, atleast with uniform sampling on sokoban. Since $\mathcal{L}_{LM}$ is more aligned with the pre-training of the base model, its effect could be stronger, thus improving generalisation beyond the training data.

Another interesting observation is that the hyperparameter τ used with $\mathcal{D}(n, \tau)$ is consistently higher for $\mathcal{L}_{LM}$, suggesting that it has a higher preference for data points in the *initial* set, which can be considered harder than other nodes.

Model Scale To test the effectiveness of our method while scaling up the LM, we demonstrate similar trends of $\mathcal{D}(n, \tau)$ outperforming $\mathcal{U}(n)$ in Table 7. We experiment with two larger models, codet5-base(220M) and codet5-large(770M). Notably, the performance of larger models is not always better than that of smaller models. This could be attributed to the fact that our experiments have been performed in the low-data regime. Studying the effects of scaling up data with parameters is left for future works. The learned heuristics with larger models are more optimal, suggesting lesser error in the predictions.

Computational Cost It is well accepted in the planning domain that a more informative heuristic is more expensive to compute (Bylander, 1994). While this holds true, even in the case of using a language model, we demonstrate that the informativeness of this learned heuristic is high enough to amortize the additional computational time. To demonstrate this, we demonstrate results on ITR, in Table 4. Recall that ITR is an averaged factor that demonstrates speed-ups in wall-clock time from the reference solution. A value > 1 implies that the solution found by the LM heuristic is faster than the base heuristic. Experiments are performed on the best models, trained with $\mathcal{D}(n, \tau)$ sampling. The ITR is $\ll 1$ for maze, which we attribute to the simplicity of the domain. The number of additional nodes searched is already quite low, therefore a reduction does not necessarily find solutions quicker. Since sokoban has a higher number of

explored nodes, the ITR on the IID test splits are much closer to 1. On the OOD split, which contains even harder puzzles, that need more iterations to solve, the LM finds solutions much faster than the inexpensive base heuristic, with an ITR > 1 . Additionally, $\mathcal{L}_{LM}$ is almost $2.5\times$ slower on average than $\mathcal{L}_{L2}$, despite the ILR being only $1.1\times$ worse. This suggests that while ϕ_{LM} is capable of learning an informative heuristic, the forward pass through the larger linear layer significantly affects efficiency.

Domain	Test Split	ITR-on-solved	ITR-on-optimal
Model : $\mathcal{L}_{L2}$			
Sokoban	IID	0.8167	0.8735
	OOD	5.9441	5.9215
Maze	IID	$5.122e-3$	$5.127e-3$
	OOD	$5.062e-3$	$5.079e-3$
Model : $\mathcal{L}_{LM}$			
Sokoban	IID	0.2626	0.2611
	OOD	2.7250	2.3978
Maze	IID	$1.958e-3$	$1.963e-3$
	OOD	$2.090e-3$	$2.096e-3$

Table 4: Speed-ups in wall clock time achieved by using the trained language model as a heuristic.

7 Conclusion

In this work, we studied the training data requirements to learn a strong heuristic for A* search to reduce the search length. This was done by disentangling the search process from language model training. In doing so, we found that the search algorithm, requires accurate prediction of heuristics for nodes nearer the goal. For enhanced generalisation of the LM heuristic, we need a greater proportion of the same set of nodes in the training data. Based on these insights, we proposed a mathematical model to quantify the contribution of nodes towards reducing the search length, and derived a distribution from which nodes can be sampled for training. This resulted in significantly better performance than uniform sampling, and in some cases outperforming heuristics trained on double the data. Moreover, we studied the computational requirements of using LM-based heuristics in A* search and find significant wall-clock speedups on harder problems. We hope such a treatment of tree-based frameworks leads to learned heuristics become more commonplace, particularly in LLM reasoning frameworks.

8 Limitations

Our study is restricted to classical puzzle domains, maze and sokoban. While we expect our domain-independent analysis to generalise to other problems, the same will need to be thoroughly evaluated. Moreover, since our work focuses on language models used as heuristics, it inherits the bias and fairness concerns associated with language models, which should be taken into consideration when deploying such models. To the best of our knowledge, there are no other negative impacts of our work.

References

- Amro Abbas, Kushal Tirumala, Dániel Simig, Surya Ganguli, and Ari S Morcos. 2023. Semdedup: Data-efficient learning at web-scale through semantic deduplication. *arXiv preprint arXiv:2303.09540*.
- Shahab Jabbari Arfaee, Sandra Zilles, and Robert C Holte. 2011. Learning heuristic functions for large state spaces. *Artificial Intelligence*, 175(16-17):2075–2098.
- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 41–48.
- Tom Bylander. 1994. The computational complexity of propositional strips planning. *Artificial Intelligence*, 69(1-2):165–204.
- Ziru Chen, Michael White, Raymond Mooney, Ali Payani, Yu Su, and Huan Sun. 2024. When is tree search useful for llm planning? it depends on the discriminator. *arXiv preprint arXiv:2402.10890*.
- Leah Chrestien, Tomas Pevny, Antonin Komenda, and Stefan Edelkamp. 2021. Heuristic search planning with deep neural networks using imitation, attention and curriculum learning. *arXiv preprint arXiv:2112.01918*.
- Gautier Dagan, Frank Keller, and Alex Lascarides. 2023. Dynamic planning with a llm. *arXiv preprint arXiv:2308.06391*.
- Alan Fern, Roni Khordon, and Prasad Tadepalli. 2011. The first learning track of the international planning competition. *Machine Learning*, 84:81–107.
- Kanishk Gandhi, Denise Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah D Goodman. 2024. Stream of search (sos): Learning to search in language. *arXiv preprint arXiv:2404.03683*.
- Alfonso Gerevini, Ivan Serina, et al. 2002. Lpg: A planner based on local search for planning graphs with action costs. In *Aips*, volume 2, pages 281–290.
- Edward Groshev, Maxwell Goldstein, Aviv Tamar, Sidharth Srivastava, and Pieter Abbeel. 2018. Learning generalized reactive policies using deep neural networks. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 28, pages 408–416.
- Lin Guan, Karthik Valmeekam, Sarath Sreedharan, and Subbarao Kambhampati. 2023. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. *Advances in Neural Information Processing Systems*, 36:79081–79094.
- Arthur Guez, Mehdi Mirza, Karol Gregor, Rishabh Kabra, Sébastien Racanière, Théophane Weber, David Raposo, Adam Santoro, Laurent Orseau, Tom Eccles, et al. 2019. An investigation of model-free planning. In *International conference on machine learning*, pages 2464–2473. PMLR.
- Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. *arXiv preprint arXiv:2305.14992*.
- Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. 1968. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4:100–107.
- Daniel Kahneman. 2011. *Thinking, fast and slow*. Farrar, Straus and Giroux.
- Daniil Kirilenko, Anton Andreychuk, Aleksandr Panov, and Konstantin Yakovlev. 2023. Transpath: Learning heuristics for grid-based pathfinding via transformers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 12436–12443.
- Levente Kocsis and Csaba Szepesvári. 2006. Bandit based monte-carlo planning. In *Machine Learning: ECML 2006*, pages 282–293, Berlin, Heidelberg. Springer Berlin Heidelberg.
- R. E. Korf. 1985. Depth-first iterative-deepening: an optimal admissible tree search. *Artificial Intelligence*, 27:97–109.
- Lucas Lehnert, Sainbayar Sukhbaatar, Paul Mcvay, Michael Rabbat, and Yuandong Tian. 2024. Beyond a*: Better planning with transformers via search dynamics bootstrapping. *arXiv preprint arXiv:2402.14083*.
- Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. 2023. Llm+ p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*.
- Adyasha Maharana and Mohit Bansal. 2022. On curriculum learning for commonsense reasoning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 983–992.

741	Max Marion, Ahmet Üstün, Luiza Pozzobon, Alex Wang, Marzieh Fadaee, and Sara Hooker. 2023. When less is more: Investigating data pruning for pretraining llms at scale. <i>arXiv preprint arXiv:2309.04564</i> .	797
742		798
743		799
744		800
745		801
746	Laurent Orseau, Marcus Hutter, and Levi HS Lelis. 2023. Levin tree search with context models. <i>arXiv preprint arXiv:2305.16945</i> .	802
747		803
748		804
749	Laurent Orseau and Levi HS Lelis. 2021. Policy-guided heuristic search with guarantees. In <i>Proceedings of the AAAI Conference on Artificial Intelligence</i> , volume 35, pages 12382–12390.	805
750		806
751		807
752		
753	Mansheej Paul, Surya Ganguli, and Gintare Karolina Dziugaite. 2021. Deep learning on a data diet: Finding important examples early in training. <i>Advances in Neural Information Processing Systems</i> , 34:20596–20607.	808
754		809
755		810
756		811
757		
758	Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. <i>Advances in Neural Information Processing Systems</i> , 36.	812
759		813
760		814
761		815
762		816
763	Ben Sorscher, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari Morcos. 2022. Beyond neural scaling laws: beating power law scaling via data pruning. <i>Advances in Neural Information Processing Systems</i> , 35:19523–19536.	817
764		818
765		819
766		820
767		821
768	David Speck, André Biedenkapp, Frank Hutter, Robert Mattmüller, and Marius Lindauer. 2021. Learning heuristic selection with dynamic algorithm configuration. In <i>Proceedings of the International Conference on Automated Planning and Scheduling</i> , volume 31, pages 597–605.	822
769		823
770		824
771		
772		
773		
774	Takeshi Takahashi, He Sun, Dong Tian, and Yebin Wang. 2019. Learning heuristic functions for mobile robot path planning using deep neural networks. In <i>Proceedings of the International Conference on Automated Planning and Scheduling</i> , volume 29, pages 764–772.	825
775		826
776		827
777		828
778		829
779		
780	Jes ús Virseda, Daniel Borrajo, and Vidal Alcázar. 2013. Learning heuristic functions for cost-based planning. <i>Planning and Learning</i> , 4.	830
781		831
782		832
783	Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. 2023. On the planning abilities of large language models-a critical investigation. <i>Advances in Neural Information Processing Systems</i> , 36:75993–76005.	833
784		834
785		
786		
787		
788	Marin Vlastelica, Anselm Paulus, Vít Musil, Georg Martius, and Michal Rolínek. 2019. Differentiation of blackbox combinatorial solvers. <i>arXiv preprint arXiv:1912.02175</i> .	
789		
790		
791		
792	Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. <i>arXiv preprint arXiv:2203.11171</i> .	
793		
794		
795		
796		
	Zhaofeng Wu, Linlu Qiu, Alexis Ross, Ekin Akyürek, Boyuan Chen, Bailin Wang, Najoung Kim, Jacob Andreas, and Yoon Kim. 2024. Reasoning or reciting? exploring the capabilities and limitations of language models through counterfactual tasks. In <i>NAACL</i> .	
	Benfeng Xu, Licheng Zhang, Zhendong Mao, Quan Wang, Hongtao Xie, and Yongdong Zhang. 2020. Curriculum learning for natural language understanding. In <i>Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics</i> , pages 6095–6104.	
	Zhun Yang, Adam Ishay, and Joohyung Lee. 2023. Coupling large language models with logic programming for robust and general reasoning from text. <i>arXiv preprint arXiv:2307.07696</i> .	
	Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2024. Tree of thoughts: Deliberate problem solving with large language models. <i>Advances in Neural Information Processing Systems</i> , 36.	
	Ryo Yonetani, Tatsunori Tanai, Mohammadamin Barekatain, Mai Nishimura, and Asako Kanezaki. 2021. Path planning using neural a* search. In <i>International conference on machine learning</i> , pages 12029–12039. PMLR.	
	Sung Wook Yoon, Alan Fern, and Robert Givan. 2006. Learning heuristic functions from relaxed plans. In <i>ICAPS</i> , volume 2, page 3.	
	Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2023a. Language agent tree search unifies reasoning acting and planning in language models. <i>arXiv preprint arXiv:2310.04406</i> .	
	Hattie Zhou, Arwen Bradley, Etai Littwin, Noam Razin, Omid Saremi, Josh Susskind, Samy Bengio, and Preetum Nakkiran. 2023b. What algorithms can transformers learn? a study in length generalization. <i>arXiv preprint arXiv:2310.16028</i> .	

A Appendix

Algorithm 1 A* Search

```

 $\mathcal{P}_{frontier} \leftarrow \{n_{start}\}$ 
 $\mathcal{P}_{closed} \leftarrow \{\}$ 
while  $|\mathcal{P}_{frontier}| > 0$  do
   $n \leftarrow \underset{n \in \mathcal{P}_{frontier}}{\operatorname{argmin}} f(n) \triangleright$  Selection
  if  $s(n) = s_{goal}$  then
    return  $n$ 
  end if
  for  $c \in \operatorname{children}(n)$  do  $\triangleright$  Expansion
     $g(c) \leftarrow g(n) + 1$ 
     $f(c) \leftarrow g(c) + h(c)$ 
    for  $m \in \mathcal{P}_{frontier} \cup \mathcal{P}_{closed}$  do
      if  $s(c) = s(m) \wedge f(c) < f(m)$  then
         $\mathcal{T} \leftarrow \mathcal{T} \cup \{c\}$ 
         $\mathcal{P}_{frontier} \leftarrow \mathcal{P}_{frontier} \cup \{c\}$ 
      end if
    end for
  end for
   $\mathcal{P}_{frontier} \leftarrow \mathcal{P}_{frontier} - \{n\}$ 
   $\mathcal{P}_{closed} \leftarrow \mathcal{P}_{closed} \cup \{n\}$ 
end while

```

A.1 Data Generation

Maze We generate mazes with a modified Prim’s algorithm⁵. The start and goal states are randomly chosen until the following criteria are met, (i) length of the optimal plan $> O_l$, (ii) ratio between length of closed set after search and length of optimal plan is $> \alpha = 3.5$. If either of these are not met within 10 tries, a new maze is generated. Criterion (i) ensures that the start and goal positions are not too close and (ii) ensures that there are sufficient number of additional expanded nodes. It serves as a surrogate for the measure of hardness $h^*(n_s)/h(n_s)$ where n_s is the start node, proposed in (Takahashi et al., 2019). The surrogate is used since it is more aligned with the chosen metrics (ILR) in this work. However, this method only creates a maze with a single path to the goal. To get multiple paths, each node is designated to either be closer to the start, or to the goal, and walls are randomly broken at the boundary of these groups⁶.

Sokoban This dataset is adapted from the open-source boxoban dataset proposed in (Guez et al., 2019). For the training puzzles, we

randomly shuffle the provided training set from the "unfiltered" split, followed by subsampling B boxes per puzzle. We use the same filters as maze, but with different hyperparameters. The IID test split uses the same criteria, but samples puzzles from the testing set of boxoban. To reduce the data creation time, we constrain the number of iterations required by A* to solve a puzzle between β_{min} and β_{max} . The OOD split is curated to contain a mix of harder puzzles with varying number of boxes, length of optimal plans and higher number of iterations. All puzzles have size 10×10 , $O_l = 20$ and $\alpha = 6$.

The exact details are in Table 5 and Table 6.

Split	# puzzles	Size	O_l
Train	750	20×20	20
Val	750	20×20	20
Test IID	500	20×20	20
Test OOD	500	30×30	30

Table 5: Dataset statistics for maze.

Split	# puzzles	B	β_{max}	β_{min}
Train	1000	2	7k	0
Val	1000	2	7k	0
Test IID	284	2	7k	0
Test OOD	15	2	14k	7k
	100	3	7k	0
	100	3	14k	7k
	100	4	7k	0
	100	4	14k	7k

Table 6: Dataset statistics for sokoban.

A.2 A* Search Algorithm

A* search is summarized in Algorithm 1.

A.3 Prompts

The language models have been trained on a regression task with context prompts, which are provided below. Since the experiments are performed with code models, we tailor the prompt accordingly. The same prompt is used for both domains, shown in Figure 4, with the puzzle representations and legend in Figure 2 for Sokoban and Figure 3.

⁵<https://github.com/john-science/mazelib>

⁶<https://stackoverflow.com/a/22308159>

```

1 #####
2 # #####
3 # # #.#
4 # . $ #
5 # $ #
6 # #####
7 #@ #####
8 # #####
9 # #####
10 #####
11 legend = "@ - player, # - wall,
           . - empty docks, ' ' - empty
           cell, $ - box, X - box on dock
           , 0 - player on dock"

```

Figure 2: Puzzle representation and legend of a training puzzle from Sokoban.

A.4 Hyperparameters

All models are trained for 40 epochs, with a learning rate of $1e - 4$, batch size of 64 and optimized with Adafactor. We implement early stopping, with the model chosen by best performance on validation MAE, computed every epoch. Training is performed on 1 A6000 Ada GPU.

A.5 Additional Results

We show additional results demonstrating the effect of sampling from $\mathcal{D}(n, \tau)$ with increasing model-scale, in [Table 7](#)

```

1 #####
2 #..@.....#
3 ###.#####.###.#####
4 #...#...#.#.#...#...#
5 #####.#.#.#.#####.#
6 #.....#.....#
7 ###.#.#.#.#.#.#.#.#.#
8 #...#.#.#.#.#...#...#
9 #.#.#.#####.#.#.#.#.#
10 #.#...#.#.#.....#
11 ###.#.#.#.#.###.#.#.#
12 #.....#...#.....#
13 #.#.#.#.#.#####.#.#.#
14 #.#.#.#.#.#.#.....#.#.#
15 #.###.#####.#.#.#.#
16 #...#.#X.....#.....#
17 #.###.#.#.#.#.#.#.#.#
18 #...#.#.#.#.#.#.#...#
19 ###.#####.###.#.###.#
20 #...#.....#...#.....#
21 #####
22 legend = "@ - player, # - wall,
           . - empty cell, X - goal"
23

```

Figure 3: Puzzle representation and legend of a training puzzle from the maze dataset.


```

1 import torch
2 def get_improved_heuristic(heuristic: int, difference: int):
3     '''
4     A function that takes in the admissible A* heuristic and adds
5     to it the difference, to return a heuristic closer to the optimal
6     cost to the goal. The difference should be calculated keeping in
7     mind the optimal cost of the puzzle.
8     '''
9     return heuristic + difference
10
11 # The difference is calculated by observing the {domain} puzzle and
12 # deducing the optimal cost to goal. The heuristic is subtracted
13 # from this optimal cost
14 # {puzzle_legend}
15 puzzle_str = "{puzzle_str}"
16 improved_heuristic = get_improved_heuristic({heuristic},

```

Figure 4: Prompt used while training the language model. {curly braces} denote a placeholder.

Base Model	Train Split	Test Split	ILR-on-solved	ILR-on-optimal	SWC	Optimal %
Maze						
Base	$\mathcal{X} \sim \mathcal{U}(n)$ $\mathcal{X} \sim \mathcal{D}(n, 2)$	IID	1.7218 1.8112	1.7245 1.8142	0.9965 0.9957	97.0 96.8
Large	$\mathcal{X} \sim \mathcal{U}(n)$ $\mathcal{X} \sim \mathcal{D}(n, 1)$		1.2963 1.6920	1.2966 1.6982	0.9995 0.9964	99.6 97.4
Base	$\mathcal{X} \sim \mathcal{U}(n)$ $\mathcal{X} \sim \mathcal{D}(n, 2)$	OOD	1.2841 1.3460	1.2722 1.3422	0.9970 0.9977	96.4 97.2
Large	$\mathcal{X} \sim \mathcal{U}(n)$ $\mathcal{X} \sim \mathcal{D}(n, 1)$		1.1531 1.3101	1.153 1.3088	0.9994 0.9980	99.2 97.6
Sokoban						
Base	$\mathcal{X} \sim \mathcal{U}(n)$ $\mathcal{X} \sim \mathcal{D}(n, 2)$	IID	10.8858 10.6828	11.1579 11.1692	0.9770 0.9791	71.83 73.94
Large	$\mathcal{X} \sim \mathcal{U}(n)$ $\mathcal{X} \sim \mathcal{D}(n, 2)$		10.3732 10.3778	10.7997 10.7343	0.9788 0.9850	74.3 80.99
Base	$\mathcal{X} \sim \mathcal{U}(n)$ $\mathcal{X} \sim \mathcal{D}(n, 2)$	OOD	14.4553 15.0611	14.4831 15.2904	0.9810 0.9828	74.70 76.39
Large	$\mathcal{X} \sim \mathcal{U}(n)$ $\mathcal{X} \sim \mathcal{D}(n, 2)$		12.8759 13.0179	12.9480 12.9534	0.9830 0.9891	76.39 83.37

Table 7: Experimental results showing the effects of the proposed sampling strategy on larger models, upto 720M parameters. Base Model refers to the variant of code-t5(base/large). Code-t5-small results have been provided in the main paper.